CONSUMER_ONBOARDING

Semgrep Consumer Onboarding Guide

Field	Value
Revision	1
Last modified	2026-06-21T00:00:00Z
Status	active
Cross-references	`Constitution.md` §11.4.166, `CLAUDE.md` §11.4.166, `scripts/semgrep/*`, `scripts/hooks/semgrep_precommit.sh`, `.docs_chain/contexts/semgrep_status.yaml`

Table of Contents

---

1. Overview

This guide walks you through integrating **semgrep** (static analysis / SAST scanning) from the Helix Constitution submodule into any consuming project.

What semgrep does for your project:

- Scans every staged file before commit for security vulnerabilities, code-quality issues, and dangerous patterns
- Blocks commits that introduce new findings
- Integrates with AI coding agents via MCP so agents can run scans autonomously
- Produces anti-bluff captured evidence per Constitution §11.4.69
- Syncs integration status via the docs_chain engine

Constitutional authority: This integration implements the universal mandate defined in `Constitution.md` §11.4.166. Every consuming project governed by this Constitution MUST install, configure, and use semgrep.

---

2. Quick Start

Complete the 3-step setup in under 2 minutes.

Step 1: Source the PATH helper

Add the semgrep PATH helper to your shell profile so semgrep is available in every session:

```
# In ~/.bashrc (or ~/.zshrc for Zsh users):
echo '. path/to/constitution/scripts/semgrep/semgrep_path.sh' >>
        ~/.bashrc

# Or with an absolute path (recommended for repeatability):
echo '[[ -f /path/to/constitution/scripts/semgrep/
        semgrep_path.sh ]] && \
    . /path/to/constitution/scripts/semgrep/semgrep_path.sh' >>
        ~/.bashrc
```

The PATH helper (`semgrep_path.sh`) adds common semgrep install directories (`~/.local/bin`, `~/.cargo/bin`, `/usr/local/bin`, `/opt/homebrew/bin`, `~/bin`) to your PATH if they are not already present. It is safe to source unconditionally -- it only adds directories that exist and never creates duplicates.

Reload your shell:

```
source ~/.bashrc
```

Step 2: Install semgrep

Run the setup script. It auto-detects whether to install via pip (fast path) or build from source (OCaml toolchain):

```
bash path/to/constitution/scripts/semgrep/semgrep_setup.sh
```

What the script does:

1. Checks if semgrep is already on PATH -- exits early if present
2. Detects available pip (`python3 -m pip`, `pip3`, or `pip`)
3. Installs semgrep via `pip install --user semgrep`
4. Verifies the binary works (`semgrep --version` returns a non-empty string)
5. Logs the result (SUCCESS/FAILED) to `docs/semgrep/Status.md`

Exit codes:

Code	Meaning
0	semgrep already installed or installation succeeded
1	pip succeeded but semgrep binary not found (bluff caught)
2	Environment problem (pip missing, network unreachable, OCaml missing)

Anti-bluff guarantee: The script does not trust `pip install` exit code 0 alone. It re-checks that `semgrep --version` produces output. A pip install that exits 0 but leaves a broken binary is caught and reported as exit 1.

Build from source (alternative):

If you prefer to build semgrep from source (requires OCaml toolchain), source the script and call `semgrep_setup_build`:

```
. path/to/constitution/scripts/semgrep/semgrep_setup.sh
semgrep_setup_build
```

The build path clones the semgrep Git repository into `submodules/semgrep/` and compiles it. Falls back to pip if the OCaml path is not supported.

Step 3: Wire the pre-commit hook

Symlink the semgrep pre-commit hook into your project's `.git/hooks/`:

```
ln -sf path/to/constitution/scripts/hooks/
        semgrep_precommit.sh .git/hooks/pre-commit
```

What the hook does:

1. Checks if semgrep is available -- skips gracefully if not installed (exit 0, no commit blocked)
2. Gets the list of staged files (`git diff --cached --name-only --diff-filter=ACM`)
3. Filters to scannable file types: `.py`, `.js`, `.ts`, `.go`, `.java`, `.kt`, `.rb`, `.c`, `.cpp`, `.h`, `.hpp`, `.yaml`, `.yml`, `.json`,`.sh`,`.bash`
4. Runs `semgrep scan --config auto --error` on the filtered files
5. If findings exist: prints output, blocks the commit (exit 1)
6. If clean: allows the commit (exit 0)
7. If semgrep errors (crash): allows the commit non-blocking (exit 0)

---

3. Automatic Post-Pull Integration

The Constitution's **post-update hook** (`scripts/post_update_hook.sh`, per §11.4.164) auto-wires semgrep when a consuming project pulls new constitution content.

After every constitution pull, the hook:

1. **Detects changed files** -- runs `git diff --name-only` against `ORIG_HEAD` to find new/modified scripts and hooks
2. **Installs changed hooks** -- copies new/modified hook scripts (including `semgrep_precommit.sh`) into `.git/hooks/` and makes them executable
3. **Validates script syntax** -- runs `bash -n` on every new/modified `.sh` file to catch syntax errors
4. **Reports summary** -- prints what was installed/updated

To trigger the hook manually after a constitution pull:

```
bash path/to/constitution/scripts/post_update_hook.sh
```

The full auto-integration sequence (clone -> pull -> ready):

```
1. git clone <project>
2. git submodule update --init constitution
3. constitution/scripts/semgrep/semgrep_setup.sh  -- installs
semgrep
4. constitution/scripts/post_update_hook.sh        -- installs
hooks, validates
5. Source semgrep_path.sh in .bashrc               -- PATH
integration
6. Run semgrep_validate.sh                         -- verify
everything works
```

The setup script (`semgrep_setup.sh`) is designed to be called from the project's own `setup.sh` (bootstrap script), so new clones get semgrep automatically.

---

4. Verification

After setup, verify semgrep is working correctly.

4.1 Quick Smoke Test

```
# Check binary is on PATH
which semgrep

# Check version
semgrep --version

# Quick scan of an empty file
echo '' | semgrep scan --config auto --json -
```

Expected: The first two commands print paths/versions. The third prints valid JSON with `"results": []`.

4.2 Full Validation

Run the validation script, which performs two checks:

```
bash path/to/constitution/scripts/semgrep/semgrep_validate.sh
```

Check 1: `semgrep_validate_check` -- Runs `semgrep scan --config auto --json` on a test fixture (a C file with a `strcpy` buffer overflow) and asserts the output is valid JSON with a `"results"` key.

Check 2: `semgrep_validate_rules` -- Verifies the semgrep rule registry is reachable by running `semgrep --config-registry --dump-config-rules` and checking for non-empty output.

Exit codes:

Code	Meaning
0	All checks PASS
1	One or more checks FAIL
2	semgrep not on PATH

Evidence is written to `docs/.semgrep/scan_<timestamp>.json` and `docs/.semgrep/registry_<timestamp>.txt`.

4.3 CI Integration Test

For a more thorough end-to-end test that exercises vulnerability detection, shell-script scanning, and produces structured evidence:

```
bash path/to/constitution/scripts/semgrep/semgrep_ci_test.sh
```

What it tests:

Test	Description	Expected
PATH check	`command -v semgrep` succeeds	PASS
Version check	`semgrep --version` returns 0	PASS
Vulnerability detection	`semgrep scan --config auto` on Python `eval()` fixture	PASS (detects >=1 finding)
Shell-script scan	`semgrep scan --config auto` on `.sh` files	PASS (no crash)

Evidence layout:

```
qa-results/
  semgrep_ci_<timestamp>/
    evidence.log              # Chronological PASS/FAIL/SKIP log
    summary.txt               # One-line summary for automation
    tmp/
      version.txt             # semgrep --version output
      vuln_test.py            # eval() test fixture
      scan_results.json       # JSON results from vulnerability
scan
      sh_scan_results.json    # JSON results from shell-script
scan
```

5. MCP Agent Wiring

5.1 Claude Code (Built-in)

Claude Code already has the Semgrep MCP built-in with three tools:

Tool	Purpose
`semgrep_scan`	Scan provided code files for security findings
`semgrep_findings`	Fetch findings from the Semgrep AppSec Platform API
`semgrep_scan_with_custom_rule`	Scan with a custom YAML rule
`semgrep_scan_supply_chain`	Scan for third-party dependency vulnerabilities
`semgrep_rule_schema`	Get the schema for writing semgrep rules

No additional configuration is needed within Claude Code. The MCP tools are available out of the box.

To verify the tools are loaded:

```
Available tools should include: semgrep_scan, semgrep_findings,
semgrep_scan_with_custom_rule, semgrep_scan_supply_chain,
semgrep_rule_schema
```

5.2 Other Agents

For agents that do not have the Semgrep MCP built-in, configure the MCP server in the agent's settings file pointing to the semgrep CLI.

Claude Code `.mcp.json` (if built-in tools not available):

```
{
  "mcpServers": {
    "semgrep": {
      "command": "semgrep",
      "args": ["scan", "--config", "auto", "--json"],
      "description": "Semgrep SAST static analysis"
    }
  }
}
```

Qwen Code (`.qwen/settings.json`):

```
{
  "mcpServers": {
```

```
    "semgrep": {
      "command": "semgrep",
      "args": ["scan", "--config", "auto", "--json"]
    }
  }
}
```

Gemini CLI (`~/.config/gemini/mcp.json`):

```
{
  "servers": {
    "semgrep": {
      "command": "semgrep",
      "args": ["scan", "--config", "auto", "--json"]
    }
  }
}
```

Note on built-in vs configured: When an agent has built-in Semgrep MCP tools (like Claude Code), those tools provide richer integration (structured tool schemas, platform API access) than a raw CLI passthrough. The CLI passthrough is sufficient for local file scanning.

---

6. docs_chain Auto-Sync

The docs_chain context at `.docs_chain/contexts/semgrep_status.yaml` keeps semgrep integration status documents in sync automatically.

Context file:

```
name: semgrep_status
description: Semgrep SAST tool integration status
sync:
  source: scripts/semgrep/semgrep_ci_test.sh
  trigger: post-pull
  exports:
    - docs/semgrep/Status.md
    - docs/semgrep/Status_Summary.md
```

How it works:

1. **Trigger:** The chain fires on `post-pull` (after a constitution sub-module update)
2. **Source:** `scripts/semgrep/semgrep_ci_test.sh` is the CI test that produces evidence
3. **Exports:** Generates/updates `docs/semgrep/Status.md` and `docs/semgrep/Status_Summary.md`

What stays in sync:

- `Status.md` -- append-only ledger of semgrep events (installs, builds, validation runs, CI test results). Both `semgrep_setup.sh` and `semgrep_validate.sh` append entries here automatically.
- `Status_Summary.md` -- operator-readable digest of integration status per §11.4.53.

To trigger a manual sync (outside the chain):

```
# Re-register the semgrep context
bash path/to/docs_chain/cmd/docs_chain/main.go sync semgrep_status
```

---

7. Anti-Bluff Protocol

The semgrep integration complies with Constitution §11.4.69 (universal sink-side positive-evidence taxonomy) and §107 (anti-bluff validation techniques).

7.1 Evidence Taxonomy

Every semgrep scan PASS must cite a captured-evidence artefact path.

Feature class	Evidence artefact	Required shape
`mediacodec_decode` (via semgrep)	`scan_results.json`	Non-empty JSON with `"results"` array
`security_scan`	`evidence.log`	Contains `PASS: <check>` lines per check
`shell_script_lint`	`sh_scan_results.json`	Valid JSON (0 or more findings)

7.2 Test Fixtures

The CI test script (`semgrep_ci_test.sh`) ships a known-vulnerable Python fixture:

```
import os

def greet_user():
    """Greet the user -- WARNING: contains eval() for CI-test
        purposes."""
    name = eval(input("Enter your name: "))  # semgrep-ignore: ci-
        test-fixture
    print("Hello, " + name)
```

```
if __name__ == "__main__":
    greet_user()
```

This fixture contains an `eval(input())` pattern that semgrep's `--config auto` ruleset detects as a security finding. If semgrep does NOT detect this, the test FAILs -- proving the analysis is working.

7.3 Self-Validation (Golden Fixtures)

The integration uses golden-good and golden-bad fixture pairs to self-validate the analyzer per §11.4.107(10).

Golden-bad fixture (`vuln_test.py` with `eval()`):

- Expected: semgrep reports >= 1 finding
- If semgrep reports 0 findings, the analyzer is broken or the registry is unreachable

Golden-good pattern (clean Python code like `x = 42`):

- Expected: semgrep reports 0 findings
- If semgrep reports findings on clean code, the ruleset is overly broad

To run the self-validation test:

```
# Replace the fixture with a clean file
echo 'x = 42' > qa-results/semgrep_ci_<ts>/tmp/vuln_test.py
bash scripts/semgrep/semgrep_ci_test.sh
# Expected: vulnerability-detection check FAILs (fixture is now
        clean)

# Restore: re-run the real script (regenerates the fixture)
bash scripts/semgrep/semgrep_ci_test.sh
# Expected: all checks PASS (includes the real eval() fixture)
```

Mutation test (paired §1.1): Remove the `eval()` call from the fixture template in `semgrep_ci_test.sh`. The CI test script's vulnerability-detection check MUST FAIL, proving the gate is not a bluff.

7.4 Validation Script

The validation script (`semgrep_validate.sh`) provides two anti-bluff checks:

1. **Scan validation** -- Pumps a real C file (`test_fixture.c`) with a known `strcpy` buffer overflow through `semgrep scan --config auto --json` and asserts the output is valid JSON with results. Uses `python3 -m json.tool` to validate JSON structure, or falls back to grepping for the `"results"` key.

2. **Registry reachability** -- Runs `semgrep --config-registry --dump-config-rules` and asserts non-empty output, confirming the semgrep registry is reachable and rules can be downloaded.

Both checks write evidence paths and produce `ab_pass_with_evidence`-style output:

```
PASS: semgrep scan produced valid JSON with results [evidence:
docs/.semgrep/scan_2026-06-21T20:27:28Z.json]
```

---

8. Troubleshooting

8.1 Semgrep Not Found

```
# Install via pip
pip install semgrep
# or
pip3 install semgrep

# Verify
which semgrep
semgrep --version

# If still not found after pip install, check ~/.local/bin is on
        PATH
echo "$PATH" | grep -q "$HOME/.local/bin" || export
        PATH="$HOME/.local/bin:$PATH"
# Add to .bashrc: echo 'export PATH="$HOME/.local/bin:$PATH"' >>
        ~/.bashrc
```

8.2 Multiple semgrep Versions

If you have semgrep installed via both pip and a system package manager, the wrong version may be on PATH first:

```
# Find all semgrep binaries
which -a semgrep

# Check which one is first
command -v semgrep

# Prioritize the correct one by adjusting PATH order in .bashrc
# Or uninstall the stale version:
pip uninstall semgrep
pip install semgrep  # reinstall fresh
```

8.3 Hook Not Firing

```
# Verify the hook is installed
ls -la .git/hooks/pre-commit

# Verify it references semgrep
grep semgrep .git/hooks/pre-commit || echo "HOOK NOT WIRED"

# If not wired, install it:
ln -sf path/to/constitution/scripts/hooks/
        semgrep_precommit.sh .git/hooks/pre-commit

# To bypass the hook for a specific commit (not recommended):
git commit --no-verify
```

8.4 MCP Integration Issues

```
# Verify semgrep is on PATH (MCP servers use PATH from the shell)
command -v semgrep
semgrep --version

# For Claude Code: check the built-in tools are loaded
# In the agent, run: semgrep_scan with a known-vulnerable file

# For configured MCP servers: verify agent settings file
cat .mcp.json | grep semgrep  # or .qwen/settings.json, etc.

# Try a direct semgrep scan from command line
semgrep scan --config auto --json /path/to/testfile.py
```

8.5 Registry Unreachable

If validation fails because the semgrep registry is unreachable (air-gapped network, proxy):

```
# Check network connectivity
curl -sI https://semgrep.dev 2>&1 | head -3

# Use a local ruleset instead of --config auto
semgrep scan --config /path/to/local/rules.yml --json testfile.py

# Or configure a proxy
export HTTP_PROXY=http://proxy:8080
export HTTPS_PROXY=http://proxy:8080
semgrep scan --config auto --json testfile.py
```

8.6 No Rules Firing

If semgrep runs but reports zero findings even on known-vulnerable code:

```
# Test with an explicit rule pattern
semgrep scan --config auto --patterns '["eval(...)"]' testfile.py

# Check registry connectivity
semgrep --config auto --dry-run /dev/null

# Update semgrep for latest rules
pip install --upgrade semgrep

# Check if --config auto is using cached rules
semgrep scan --config auto --json --no-rewrite-rule-ids
        testfile.py
```

---

9. File Reference

File	Purpose	Inherited
`scripts/semgrep/semgrep_setup.sh`	Install semgrep (pip or OCaml build). Auto-detects available methods.	By reference
`scripts/semgrep/semgrep_path.sh`	PATH integration for `.bashrc`/`.zshrc`. Adds common semgrep install locations.	By reference
`scripts/semgrep/semgrep_validate.sh`	Anti-bluff validation: scan + registry checks with evidence output.	By reference
`scripts/semgrep/semgrep_ci_test.sh`	CI-ready integration test. 4 checks, JSON evidence, golden fixtures.	By reference
`scripts/hooks/semgrep_precommit.sh`	Pre-commit hook. Scans staged files, blocks on findings. Graceful skip if semgrep absent.	By reference
`scripts/post_update_hook.sh`	Post-pull auto-propagation hook. Installs hooks, validates scripts (§11.4.164).	By reference
`.docs_chain/contexts/semgrep_status.yaml`	docs_chain context for auto-syncing status documents.	By reference

File	Purpose	Inherited
`docs/semgrep/Status.md`	Append-only event ledger. Automatically updated by setup and validation scripts.	Generated
`docs/semgrep/Status_Summary.md`	Operator-readable integration status digest.	Generated
`docs/semgrep/VERIFICATION.md`	Full verification protocol with per-step evidence requirements.	By reference
`docs/semgrep/CONSUMER_ONBOARDING.md`	This guide.	By reference
`docs/.semgrep/test_fixture.c`	C file with `strcpy` buffer overflow used by validation script.	Generated (by validate)
`qa-results/semgrep_ci_<ts>/`	Per-run evidence directory from CI test.	Generated

Inheritance pattern: All scripts in `scripts/semgrep/` and `scripts/hooks/semgrep_precommit.sh` are inherited by reference per §11.4.28. Consuming projects NEVER copy them -- they reference the canonical paths in the constitution submodule. The post-update hook (`scripts/post_update_hook.sh`) detects changes and installs hooks automatically.